



ARAB ACADEMY FOR SCIENCE & TECHNOLOGY & MARITIME TRANSPORT
COLLEGE OF ENGINEERING & TECHNOLOGY

Course	CC410 Systems Programming
Lecturer	Dr. Sherine Nagy Saleh – Dr. Noha Seddik Abdelsalam
TA	Eng. Hoda Elkhodery – Eng. Eman Maher – Eng. Ghada Aslan

MODIFIED SIC/XE ASSEMBLER PROJECT

Create a program that emulates a modified version of the SIC/XE machine. This simulation should include both the first and second passes of assembly, and generate HTME records for the modified SIC/XE architecture.

Introduction to modi-SIC/XE

- Consists of the original SIC/XE instruction set.
- Utilizes the same data management techniques with BYTE, WORD, RESB, and RESW.
- Preserves the existing addressing modes found in SIC/XE.
- Introduces a new instruction format, **Format 4F**, which enables conditional execution of instructions. This new format eliminates the need for separate branching instructions, such as JLT, by allowing instructions to execute or be skipped based on specific condition flags.
- Adds support for **Blocks, with slight modifications**, enabling the program to be divided into up to four blocks:

1. DEFAULT block, containing executable instructions in formats 1 and 2.
2. DEFAULTB block, encompassing executable instructions in formats 3, 4, and 4F.
3. CDATA block, designated for smaller-sized data.
4. CBLKS block, intended for larger memory blocks.

Modi-SIC/XE Implementation Details

It takes as an input a text file (in.txt) that contains an assembly program written in modi-SIC/XE assembly language containing comments, number of lines.

Generating Intermediate File

Develop a program that processes a text file (in.txt), which includes an assembly program written in the modified SIC/XE assembly language. This file can also include comments. The primary task is to generate an intermediate file.

Pass 1 Objective: Process the initial input file to determine the memory allocation and create a symbol table.

- **Task:** Read the provided input file (potentially "in.txt" before cleaning to "intermediate.txt") to compute and track the location counter (LC) for each program line. Simultaneously, identify and catalog symbols (labels, constants, etc.) to generate a symbol table.
- **Outputs:**
 - **out_pass1.txt:** This file should list each line's corresponding LC, showing where in memory each instruction or variable is allocated.
 - **symbTable.txt:** This file should include the symbol table with entries for all labels or symbols in the program mapped to their respective LC values.

Pass 2 Objective: Convert assembly instructions into machine code and format the output for program loading and execution.

- **Task:** Read the "intermediate.txt" file, which contains cleaned assembly instructions without comments or line numbers. Generate object code for each line and compile the output into the HTME format (Header, Text, Modification, End records).
- **Outputs:**
 - **out_pass2.txt:** This file should contain the object code for each line of the program, corresponding to the machine-readable instructions generated from the assembly code.
 - **HTME.txt:** This file should organize the object code into the HTME format, necessary for loading the program into memory and execution, detailing segments for initialization, execution code, modifications for relocatability, and the end record signaling the end of the object program.

Notes

- The assembler is designed to handle literals within the source code. It automatically identifies literals used in the instructions, allocates memory for them at the end of the pass, and assigns appropriate addresses.
- Up to four distinct memory blocks may be utilized to segment the program logically: DEFAULT, DEFAULTB, CDATA, and CBLKS. Although the architecture allows multiple blocks for organizing code and data, the entire program may consist of only 1,2 or 3.
- Your code should also include error handling to detect
 - **Unidentified Block Names:** The assembler should validate block names used in the assembly code against the predefined block names. Any reference to blocks outside of the specified names (DEFAULT, DEFAULTB, CDATA, CBLKS) should trigger an error, indicating an unrecognized block name.
 - **Unidentified Symbols:** Proper error messages should be generated for symbols that are referenced but not defined or are outside the scope of their declaration. This check ensures all symbols used in the program are correctly declared and accessible within their defined blocks.

Modi-SIC/XE Assessment Details

A maximum of 3 students per group is allowed, each team member will be assessed for his individual work as well as for the group work. group members should agree on the programming languages and code structures to be used. The table below shows the tasks required from each team member.

Student 1	Student 2	Student 3
Input file processing, Intermediate file generation	Pass1: Location counter management	Pass1: Symbol table generation
Object code: format 3,4	Object code: format 1,2	Object code: format 4F
Handling memory reservations (Byte, Word, RESB, etc...) Literals	Blocks	HTME records generation Error handling

Each team should submit the merged source code and executable files and each student should submit the individual source code, executable files, and a report that includes:

- example statements on how to use your program (both individual and merged). E.g. for a python program: python3 code.py --data in.txt
- design issues and sample run

Format 4F Layout (4 bytes)

It's written in code as: CADD A, BUFFER, Z

6 bits	4 bits	2 bits	20 bits
opcode	Register	Condition Flag	Address

Opcode (6 bits): Specifies the operation to be performed (e.g., ADD, SUB, etc.).

Register (4 bits): Specifies the target register (e.g., A, X, T).

Condition Flag (2 bits): Determines the conditions under which the instruction should be executed.

Possible values could include:

- 00: If the Zero (Z) flag is set.
- 01: If the Negative (N) flag is set.
- 10: If the Carry (C) flag is set.
- 11: If the Overflow (V) flag is set.

Address (20 bits): Specifies the memory address.

⇒ This format does **not** support immediate or indirect addressing modes.

Instruction set of Format 4F

Mnemonic	Format	Opcode
CADD R, M, F	4F	BC
CSUB R, M, F	4F	8C
CLOAD R, M, F	4F	E4
CSTORE R, M, F	4F	FC
CJUMP M, F	4F	CC